

Universidad Libre De Colombia

Facultad De Ingeniería

Manual de Instalación

Proyecto 4: Sistema de Gestión de Mascotas para una Clínica Veterinaria
Despliegue y ejecución del sistema

Presentado por:
Santiago Enrique Arquez Cervantes, Cristian David Diaz Alarcón, Ginneth Natalia González Puentes, Doncel Castro Stephania

Docente:
Miguel Hernández Bejarano

Bogotá D.C.

Introducción

Este manual explica de manera breve cómo instalar, ejecutar y comprobar el sistema de gestión de mascotas para una clínica veterinaria. El sistema se compone de tres microservicios hechos con Java 21 y Spring Boot (Owner-Service, Pet-Service y Appointment-Service), cada uno con su propia base de datos MySQL, e implementado con Docker Compose. En este orden de ideas; existen tres maneras de correr el proyecto: con Docker en tu propio computador, con Docker en GitHub Codespaces (un entorno en la nube) o de forma local sin Docker.

1. Requisitos

Herramienta	Versión	Para qué se usa
Docker Desktop (con Docker Compose v2)	Última estable	Para levantar los microservicios y las bases de datos (la forma recomendada).
Git	Última estable	Para clonar y subir el repositorio.
Java JDK	21 LTS (o superior)	Para compilar, correr las pruebas automáticas y ejecutar el sistema en local sin Docker.
Maven	3.9 o superior	Para compilar el proyecto y correr las pruebas.
Postman	Última estable	Para ejecutar la colección de pruebas de la API (opcional).
Navegador web	Cualquiera reciente	Para consultar la documentación de Swagger.

Para correr el sistema con Docker, no se necesita instalar Java ni Maven, porque la compilación ocurre dentro de los contenedores. Eso sí, conviene tener un equipo con al menos 8 GB de memoria RAM, ya que se ejecutan seis contenedores a la vez (tres microservicios y tres bases de datos).

2. Obtención del Proyecto

Paso_1. Clonar el repositorio público del proyecto (https://github.com/SantiagoArquez/Veterianrias) o descárgalo como ZIP desde GitHub con Code > Download ZIP:

```
git clone https://github.com/SantiagoArquez/Veterianrias.git
cd Veterianrias
```

Paso_2. Revisar que en la carpeta raíz este el archivo docker-compose.yml y las carpetas de los tres servicios:

```
ls
# docker-compose.yml  owner-service  pet-service  appointment-service  postman
README.md
```

Nota: *todos los comandos de docker compose hay que ejecutarlos en la carpeta donde está docker-compose.yml. Si los corres en otra carpeta, te sale el mensaje "no configuration file provided: not found".*

Estructura del proyecto

```
Veterinarias/
 ├── owner-service/        (puerto 8081)
 ├── pet-service/          (puerto 8082)
 ├── appointment-service/   (puerto 8083)
|    └── cada servicio: pom.xml, Dockerfile, src/main, src/test
 ├── postman/              (colección de Postman)
 ├── docker-compose.yml
└── README.md
```

3. Despliegue con Docker Compose

Paso 1. Abrir Docker Desktop y esperar a que indique que el motor está corriendo.

Paso 2. En la carpeta raíz del proyecto ejecutar:

```
docker compose up --build
```

La primera vez tarda varios minutos, porque tiene que descargar las imágenes de Maven, Java y MySQL y compilar los tres servicios. Las siguientes veces es mucho más rápido.

Paso 3. Espera a que cada servicio muestre en los registros un mensaje como este (cada uno tarda más o menos 35 segundos en arrancar):

```
owner-service  | ... Tomcat started on port 8081 (http) with context path '/'
pet-service    | ... Tomcat started on port 8082 (http) with context path '/'
appointment-service | ... Tomcat started on port 8083 (http) with context path '/'
```

Paso 4. Mira el estado de los contenedores desde otra terminal:

```
docker compose ps
```

Tienen que aparecer seis contenedores: mysql-owners, mysql-pets, mysql-appointments, owner-service, pet-service y appointment-service.

4. Direcciones del Sistema

Servicio	Puerto	Documentación Swagger
Owner-Service	8081	http://localhost:8081/swagger-ui.html
Pet-Service	8082	http://localhost:8082/swagger-ui.html
Appointment-Service	8083	http://localhost:8083/swagger-ui.html
MySQL de propietarios	3307 (equipo) → 3306 (contenedor)	Base owners_db
MySQL de mascotas	3308 (equipo) → 3306 (contenedor)	Base pets_db
MySQL de citas	3309 (equipo) → 3306 (contenedor)	Base appointments_db

Detener el sistema

```
# Detener y eliminar los contenedores (los datos se conservan en volúmenes)
docker compose down

# Detener y además BORRAR todos los datos de las bases
docker compose down -v
```

Reiniciar un solo servicio

```
docker compose stop owner-service
docker compose start owner-service
```

Después de iniciar un servicio, espera unos 35 segundos antes de usarlo, mientras la aplicación termina de arrancar.

5. Despliegue en GitHub Codespaces

Si no tiene Docker en su equipo, puede correr el proyecto en GitHub Codespaces, un entorno de desarrollo en la nube que ya trae Docker, Java y Maven. Necesitará una cuenta de GitHub; la gratuita incluye cierta cantidad de horas al mes, y conviene revisar el cupo vigente en la página de facturación de GitHub.

Paso 1. Es necesario que tenga el proyecto en un repositorio de GitHub. El de este proyecto es https://github.com/SantiagoArquez/Veterianrias; si trabaja con una copia, suba el contenido arrastrando los archivos desde la opción "uploading an existing file" o con git push.

Paso 2. En el repositorio pulse el botón verde Code, abra la pestaña Codespaces y elija Create codespace on main.

Paso 3. En la terminal del Codespace, ubique la carpeta donde está docker-compose.yml y ejecute:

```
docker compose up --build
```

Paso 4. Abra la pestaña PORTS (junto a la terminal), busque los puertos 8081, 8082 y 8083 y pulse el ícono del globo para abrirlos en el navegador, añadiendo /swagger-ui.html al final de la dirección.

Paso 5. Si quiere probar desde la terminal, abra una segunda con el botón + y use curl contra localhost (mira la sección 8).

Paso 6. Cuando termine, ejecute docker compose down y detenga el Codespace desde github.com/codespaces con la opción Stop codespace. Ojo: si solo cierra la pestaña del navegador, el Codespace sigue gastando horas.

6. Ejecución local sin Docker

Esta opción sirve para depurar un servicio por separado. Se necesita Java 21 o superior, Maven y un servidor MySQL 8 funcionando.

Paso 1. Tenga MySQL escuchando en localhost:3306 con usuario root y contraseña root (los valores por defecto). Las bases owners_db, pets_db y appointments_db se crean solas cuando arranca cada servicio.

Paso 2. Abra tres terminales, una por servicio, y en cada una ejecute:

```
cd owner-service && mvn spring-boot:run
cd pet-service && mvn spring-boot:run
cd appointment-service && mvn spring-boot:run
```

Paso 3. Si su MySQL tiene otras credenciales o los servicios están en otras direcciones, defina las variables de entorno de la sección 10 antes de arrancar.

7. Ejecución de las pruebas automáticas

Las pruebas no necesitan Docker ni MySQL: usan una base H2 en memoria y simulan las llamadas entre servicios. Se corren una vez por cada servicio:

```
cd owner-service && mvn test
cd ../pet-service && mvn test
cd ../appointment-service && mvn test
```

Cada ejecución debe terminar con BUILD SUCCESS y con la línea "Tests run: N, Failures: 0, Errors: 0". Lo esperado son 18 pruebas en Owner-Service, 15 en Pet-Service y 16 en Appointment-Service, 49 en total. La primera vez Maven descarga las dependencias y puede demorar varios minutos.

Nota: *Los archivos pom.xml traen la propiedad argLine con -Dnet.bytebuddy.experimental=true. Hace falta cuando tiene instalado un JDK más nuevo que Java 21 (por ejemplo Java 25), porque la versión de Byte Buddy que usa Mockito solo reconoce oficialmente hasta Java 23. No afecta a la aplicación.*

8. Verificación de la instalación

Con los tres servicios corriendo, puede comprobar el sistema con los cinco casos de integración del enunciado. Junto a cada caso se indica la respuesta esperada.

Caso 1. Registrar propietario (201 Created):

```
curl -i -X POST localhost:8081/owners -H "Content-Type: application/json" \
  -d
'{"nombre":"Ana","apellido":"Pérez","documento":"1001","telefono":"3001234567","correo":"
ana@mail.com","direccion":"Calle 1"}'
```

Caso 2. Registrar mascota asociada a un propietario (201 Created):

```
curl -i -X POST localhost:8082/pets -H "Content-Type: application/json" \
  -d
'{"nombre":"Firulais","especie":"Perro","raza":"Labrador","sexo":"MACHO","fechaNacimiento
":"2020-05-01","peso":25.5,"color":"Negro","ownerId":1}'
```

Caso 3. Programar una cita (201 Created):

```
curl -i -X POST localhost:8083/appointments -H "Content-Type: application/json" \
  -d '{"petId":1,"fecha":"2030-01-15","hora":"10:30","motivoConsulta":"Vacunación"}'
```

Caso 4. Cita para una mascota inexistente (404 Not Found):

```
curl -i -X POST localhost:8083/appointments -H "Content-Type: application/json" \
  -d '{"petId":999,"fecha":"2030-01-15","hora":"11:00","motivoConsulta":"Control"}'
```

Caso 5. Mascota para un propietario inexistente (400 Bad Request):

```
curl -i -X POST localhost:8082/pets -H "Content-Type: application/json" \
  -d
'{"nombre":"Rex","especie":"Perro","raza":"Criollo","sexo":"MACHO","fechaNacimiento":"202
1-01-01","peso":10,"ownerId":999}'
```

Consulta de cita completa (200 OK):

```
curl -i localhost:8083/appointments/1/completa
```

Servicio no disponible (503 Service Unavailable):

```
docker compose stop owner-service
curl -i -X POST localhost:8082/pets -H "Content-Type: application/json" \
  -d
'{"nombre":"Luna","especie":"Gato","raza":"Siamés","sexo":"HEMBRA","fechaNacimiento":"202
2-01-01","peso":4,"ownerId":1}'
docker compose start owner-service
```

Ejecute los casos en ese orden, porque los siguientes dependen de los registros que crean los anteriores (propietario 1, mascota 1 y cita 1).

9. Colección de Postman

La carpeta postman trae una colección que verifica toda la API de forma automática (28 peticiones). La puede correr con Newman desde la terminal o con la aplicación de escritorio de Postman.

9.1 Con Newman (recomendado)

Paso 1. Empiece con bases de datos vacías, para que los datos de la colección no choquen con registros anteriores:

```
docker compose down -v
docker compose up --build -d
```

Paso 2. Espere unos 40 segundos a que arranquen los tres servicios (lo puede comprobar con docker compose ps) y ejecute la colección desde la carpeta raíz del proyecto:

```
npx newman run postman/Clinica_Veterinaria.postman_collection.json
```

La primera vez, npx pide confirmar la instalación de newman y responde y. Se necesita Node.js, que en GitHub Codespaces ya viene instalado. Lo esperado es una lista de comprobaciones ✓ y una tabla final con 28 peticiones y 28 comprobaciones, con 0 fallos.

Nota: *la colección deja una cita ATENDIDA que ocupa un horario, así que no se puede correr dos veces sobre los mismos datos. Para repetirla, reinicie las bases con docker compose down -v y docker compose up --build -d.*

9.2 Con la aplicación de escritorio de Postman

Paso 1. En Postman elija Import y seleccione el archivo postman/Clinica_Veterinaria.postman_collection.json.

Paso 2. Si los servicios corren en el mismo equipo, no tiene que cambiar nada: las variables owner_url, pet_url y appointment_url apuntan a localhost por defecto. Si corren en GitHub Codespaces, descarga el archivo de la colección, haz públicos los puertos 8081, 8082 y 8083 (pestaña PORTS, clic derecho, Visibilidad del puerto) y reemplace las tres variables por las direcciones reenviadas de cada puerto. Cuando termine, vuelva a poner los puertos como privados.

Paso 3. Haga clic en la colección, elija Run collection y respete el orden que ya trae. La colección guarda sola los identificadores que se van creando en las variables owner_id, pet_id y appointment_id, y revise el código de estado de cada petición.

10. Variables de configuración

Cada servicio lee su configuración del archivo application.yml, pero puede cambiar cualquier valor con una variable de entorno. Docker Compose ya define las que se necesitan para la red de contenedores.

Variable	Valor por defecto	Descripción
DB_URL	jdbc:mysql://localhost:3306/<base>	Cadena de conexión JDBC hacia MySQL.
DB_USER	root	Usuario de la base de datos.
DB_PASSWORD	root	Contraseña de la base de datos.
PET_SERVICE_URL	http://localhost:8082	Dirección de Pet-Service, que usan Owner y Appointment.
OWNER_SERVICE_URL	http://localhost:8081	Dirección de Owner-Service, que usa Pet.
CLIENT_CONNECT_TIMEOUT_MS	2000	Tiempo máximo de espera para conectarse con otro servicio.
CLIENT_READ_TIMEOUT_MS	5000	Tiempo máximo de espera para recibir la respuesta de otro servicio.
MYSQL_ROOT_PASSWORD	root	Contraseña de root de los contenedores MySQL; la puedes definir en un archivo .env junto a docker-compose.yml.

11. Solución de problemas

Síntoma	Causa y solución
Aparece "no configuration file provided: not found" al ejecutar docker compose	Ejecutaste el comando fuera de la carpeta que tiene docker-compose.yml. Entra a esa carpeta con cd y vuelve a intentarlo.
Un puerto (8081, 8082, 8083, 3307, 3308 o 3309) ya está en uso	Cierra el programa que lo está usando o cambia el puerto del equipo en docker-compose.yml (el número de la izquierda en "ports").
Falla el arranque por la conexión a MySQL	MySQL todavía se está inicializando. Los servicios esperan a que la base esté saludable y se reinician solos; si el problema sigue, revisa con docker compose logs <servicio>.
Respuesta 503 al crear una mascota o una cita	El servicio remoto está detenido o todavía arrancando. Revisa con docker compose ps y espera unos 35 segundos después de iniciarlo.
Respuesta 404 "La ruta solicitada no existe" al abrir http://localhost:8081/	Es normal: la raíz no tiene contenido. Usa /swagger-ui.html o los endpoints /owners, /pets y /appointments.
Las pruebas fallan con "Java 25 is not supported by the current version of Byte Buddy"	Comprueba que el pom.xml tenga la propiedad argLine con -Dnet.bytebuddy.experimental=true, o ejecuta mvn test -DargLine="-Dnet.bytebuddy.experimental=true".
La primera compilación va muy lenta	Es normal: se están descargando las dependencias de Maven y las imágenes de Docker. Las siguientes veces usa la caché.
Quieres empezar con las bases de datos vacías	Ejecuta docker compose down -v y después docker compose up --build.